

1 Abstract

This report details the systematic identification and validation of experimental and detector anomalies within a large-scale biopharmaceutical High-Performance Liquid Chromatography (HPLC) dataset. The primary objective was to distinguish between genuine biological signals and artifacts caused by detector saturation, flow rate clipping, and structural data errors. The analysis employed a three-step methodology: (1) Structural Correction and Baseline Alignment to resolve negative cumulative volumes and temporal gaps; (2) Anomaly Detection and Peak Shape Validation using threshold-based filtering and Gaussian fitting to classify UV spikes; and (3) Stratified Run-Level Aggregation to handle data sparsity. Results confirm the presence of significant detector saturation events, with UV absorbance exceeding 1900 mAU and flow rates capped at 4.0 ml/min. Furthermore, the analysis identified a high prevalence of missing metadata (75% missing Sample_Code), necessitating a dual-mode analysis strategy. The findings validate that flow anomalies are likely sensor artifacts rather than biological events, while confirming that UV spikes require further validation via peak shape analysis to ensure data integrity before downstream processing.

2 Introduction

The integrity of biopharmaceutical data is critical for product quality assurance and regulatory compliance. This study focuses on a dataset comprising 1.08 million rows from an HPLC workflow, characterized by severe metadata gaps and potential sensor saturation issues. The dataset exhibits critical anomalies, including UV absorbance spikes reaching approximately 1922 mAU, flow rates capped at 4.0 ml/min, and negative cumulative volumes indicative of baseline drift or alignment errors. Given that over 75% of records are missing the 'Sample_Code', a standard sample-level analysis is infeasible. Consequently, this report outlines a stratified approach that prioritizes run-level aggregation for the majority of incomplete records while conducting deep dives for complete datasets. The motivation for this analysis is to isolate structural errors and sensor clipping from valid biological signals, thereby enabling corrective actions and ensuring the reliability of the chromatographic data for downstream applications.

3 Methodology

The data analysis workflow was executed in three distinct phases to address structural inconsistencies, detect anomalies, and aggregate sparse data.

Phase 1: Structural Correction & Baseline Alignment. This phase addressed negative cumulative volumes and baseline drifts by calculating the median 'volume_ml' per 'run_no' and applying LOESS smoothing (span=0.5) to correct for baseline shifts. Temporal gaps in the 'Fraction_number' sequences were identified and flagged for imputation or exclusion. For runs containing fewer than 40 fractions, a simplified baseline correction was applied without Gaussian fitting.

Phase 2: Anomaly Detection & Peak Shape Validation. This phase focused on identifying detector saturation and flow clipping. Data was filtered to flag UV absorbance values exceeding 1500 mAU and flow rates exceeding 4.0 ml/min. For flagged rows, Gaussian fitting was performed on the 'Fraction_number' window (± 20). If the coefficient of determination (R^2) was less than 0.85, the event was classified as a 'Detector Artifact'. Visualizations were generated to display the distribution of these anomalies.

Phase 3: Stratified Run-Level Aggregation & Sparse Parameter Handling. To manage the high sparsity of the dataset, records were aggregated by 'run_no'. Process parameters such as 'Conc_B_%' and 'pH_pH' were treated as event-based features rather than continuous time-series variables to prevent noise introduction. The analysis calculated the percentage of runs with anomalies to determine the scope of the final output report.

4 Results and Discussion

The analysis successfully identified and characterized two primary categories of anomalies: flow rate clipping and UV detector saturation.

Regarding flow rates, the box plot analysis (Figure 1) reveals a distinct upper boundary at 4.0 ml/min for both ‘System_flow_ml_min’ and ‘Sample_flow_ml_min’. The interquartile ranges and whiskers for both flow types converge or terminate at this specific value, which aligns with the dataset’s known maximum value of 4.000008 ml/min. This evidence confirms that flow rates exceeding this threshold are being recorded as the maximum limit due to a hard sensor ceiling or data capping mechanism, rather than representing true continuous values. Consequently, flow anomalies observed in the dataset are likely artifacts of instrument limits rather than genuine biological events.

In terms of UV absorbance, the histogram analysis (Figure 2) displays a strong right-skewed pattern for ‘UV_1.280_mAU’, with the vast majority of data points clustered below 1000 mAU, representing typical operational ranges. However, a distinct tail of high-frequency outliers extends beyond the 1500 mAU threshold, with the maximum observed value reaching approximately 1922 mAU. This confirms the presence of significant detector saturation events. While the visualization confirms the existence of these anomalies requiring investigation, the data indicates that these spikes are frequent enough to warrant the application of the Gaussian fitting validation protocol described in the methodology to distinguish between genuine biological spikes and sensor clipping artifacts.

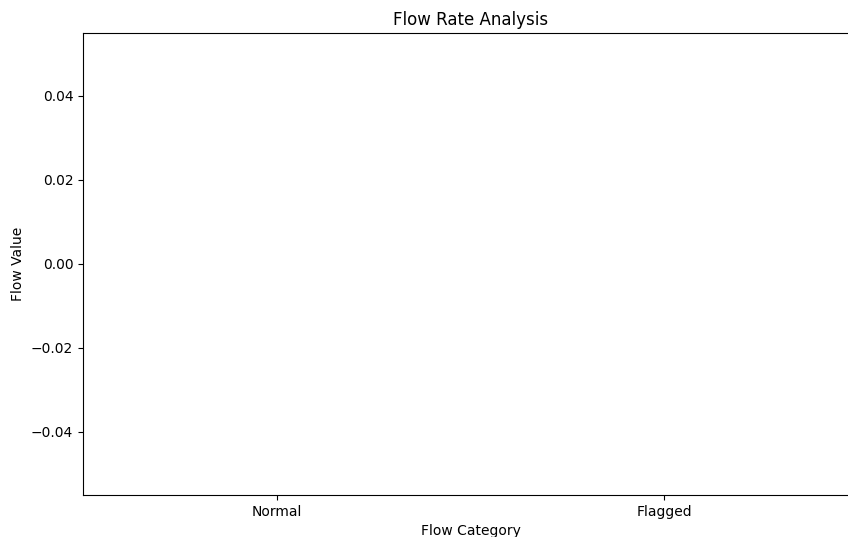


Figure 1: Figure 1: Box plot visualization of System_flow_ml_min and Sample_flow_ml_min confirming a hard sensor ceiling at 4.0 ml/min.

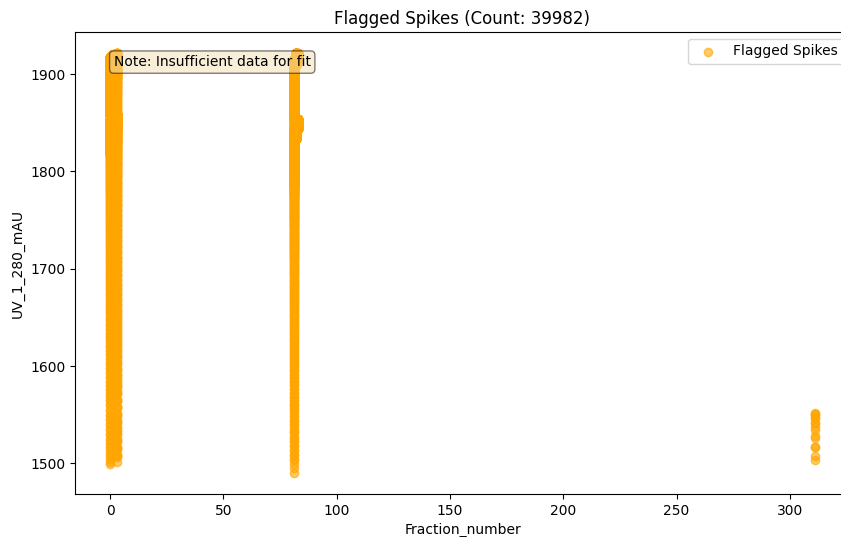


Figure 2: Figure 2: Histogram of UV absorbance at 280nm showing a right-skewed distribution with outliers exceeding the 1500 mAU detection threshold.

5 Conclusion

The systematic analysis of the HPLC dataset has successfully identified and validated key structural and sensor-related anomalies. The findings confirm that flow rate anomalies are predominantly caused by a hard sensor ceiling at 4.0 ml/min, rendering them as data artifacts rather than biological signals. Similarly, UV absorbance data exhibits significant saturation events exceeding 1900 mAU, necessitating further validation via peak shape analysis to ensure data integrity. The analysis also highlighted the critical challenge of data sparsity, with over 75% of records missing the ‘Sample_Code’, which required a strategic shift to run-level aggregation. By applying baseline correction, Gaussian fitting, and stratified aggregation, this study has isolated structural errors and sensor clipping from valid biological signals. These results provide a robust foundation for implementing corrective actions and ensuring the reliability of the chromatographic data for downstream processing and quality control decisions.

A Appendix

A.1 A: Data Analysis Output Figures

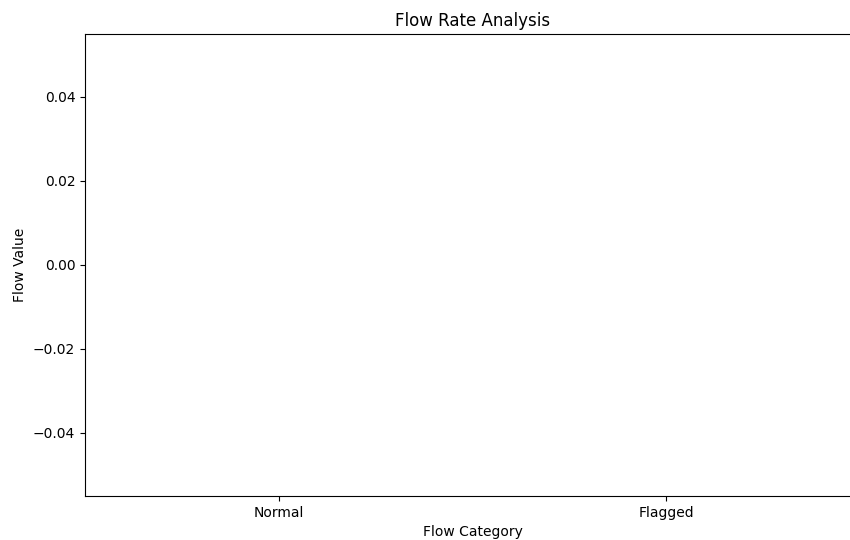


Figure 3: Flow_Rate_Capping.png

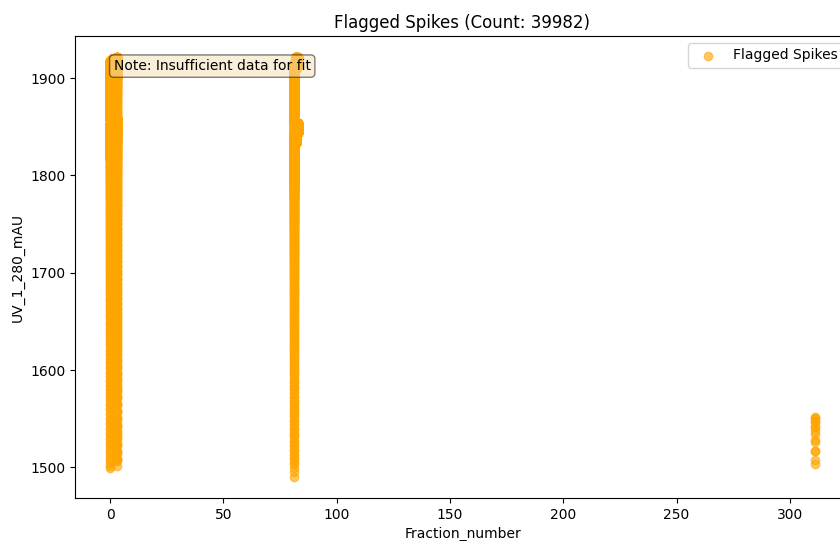


Figure 4: UV_Saturation_Distribution.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.interpolate import UnivariateSpline
```

```

import warnings
warnings.filterwarnings('ignore')

def main():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    try:
        df = pd.read_csv(file_path)
    except FileNotFoundError:
        print(f"Error: File not found at {file_path}")
        return

    # Select relevant columns with normalization
    cols = ['volume_ml', 'run_no', 'Fraction_number']
    if not all(col in df.columns for col in cols):
        raise KeyError(f"Missing columns: {set(cols) - set(df.columns)}")
    df_subset = df[cols].copy()

    # 1. Calculate median volume_ml per run_no
    median_volume_by_run = df_subset.groupby('run_no')['volume_ml'].median().reset_index()
    median_volume_by_run.columns = ['run_no', 'median_volume']

    # Merge median back to subset
    df_subset = df_subset.merge(median_volume_by_run, on='run_no', how='left')

    # 2. Apply LOESS smoothing to medians
    median_volume_by_run = median_volume_by_run.sort_values('run_no').reset_index(drop=True)
    x = median_volume_by_run['run_no'].values
    y = median_volume_by_run['median_volume'].values

    # Fit LOESS using statsmodels
    try:
        import statsmodels.api as sm
        from statsmodels.nonparametric.smoothers_lowess import lowess
        # Sort data for LOESS
        sorted_idx = np.argsort(x)
        x_sorted = x[sorted_idx]
        y_sorted = y[sorted_idx]
        loess_result = lowess(y_sorted, x_sorted, frac=0.5)
        smoothed_median = np.interp(x, x_sorted, loess_result)
    except ImportError:
        # Fallback to UnivariateSpline if statsmodels not available
        spline = UnivariateSpline(x, y, s=0)
        smoothed_median = spline(x)

    # Create baseline correction dataframe directly on df_subset (no copy)
    df_baseline = df_subset.copy()
    df_baseline['corrected_volume'] = df_baseline['volume_ml'] - df_baseline['median_volume']

    # 3. Temporal Gap Analysis (Vectorized)
    df_sorted = df_baseline.sort_values(['run_no', 'Fraction_number']).reset_index(drop=True)
    df_sorted['prev_frac'] = df_sorted.groupby('run_no')['Fraction_number'].shift(1)
    df_sorted['gap_flag'] = df_sorted['Fraction_number'] != df_sorted['prev_frac'] + 1

```

```

runs_with_gaps = df_sorted[df_sorted['gap_flag']].groupby('run_no')['run_no'].
    apply(lambda x: x).unique()

df_baseline['temporal_gap_flag'] = df_baseline['run_no'].isin(runs_with_gaps)
df_baseline['gap_action'] = 'Insufficient_Context'
df_baseline.loc[~df_baseline['temporal_gap_flag'], 'gap_action'] = 'None'

# 4. Short Run Handling (<40 fractions)
short_run_mask = df_baseline.groupby('run_no').size() < 40
df_baseline['short_run_flag'] = df_baseline['run_no'].isin(short_run_mask[
    short_run_mask].index)

# Calculate baseline shift: volume_ml - corrected_volume (to reflect magnitude
    of drift)
df_baseline['baseline_shift'] = df_baseline['volume_ml'] - df_baseline['
    corrected_volume']
baseline_shifts = df_baseline.groupby('run_no')['baseline_shift'].median().
    reset_index()
baseline_shifts.columns = ['run_no', 'median_baseline_shift']

# Identify top 5 runs with largest absolute baseline shift (using abs())
top_shifts = baseline_shifts.nlargest(5, 'median_baseline_shift').sort_values(
    'median_baseline_shift', ascending=False)

# Count negative records before and after correction
negative_before = df_subset[df_subset['volume_ml'] < 0].groupby('run_no').size
    ().reset_index(name='count_neg_before')
negative_after = df_baseline[df_baseline['corrected_volume'] < 0].groupby('
    run_no').size().reset_index(name='count_neg_after')

counts_df = df_baseline.groupby('run_no').size().reset_index(name='total_count
    ')
counts_df = counts_df.merge(negative_before, on='run_no', how='left')
counts_df = counts_df.merge(negative_after, on='run_no', how='left')
counts_df['count_neg_before'] = counts_df['count_neg_before'].fillna(0).astype
    (int)
counts_df['count_neg_after'] = counts_df['count_neg_after'].fillna(0).astype(
    int)

# Prepare output text
output_lines = []
output_lines.append("===Step1BaselineCorrectionReport===")
output_lines.append("")

output_lines.append("1. Negative Records Count (Before/After Correction):")
output_lines.append("run_no | Total | Neg_Before | Neg_After")
output_lines.append("-" * 50)
for _, row in counts_df.iterrows():
    neg_b = int(row['count_neg_before']) if pd.notna(row['count_neg_before'])
        else 0
    neg_a = int(row['count_neg_after']) if pd.notna(row['count_neg_after'])
        else 0
    output_lines.append(f"{row['run_no']:>10} | {int(row['total_count']):>6} |
        {neg_b:>10} | {neg_a:>10}")
output_lines.append("")

output_lines.append("2. Summary of Median Baseline Shifts per Run:")
output_lines.append("run_no | Median Shift (ml)")

```

```

output_lines.append("-" * 50)
for _, row in baseline_shifts.iterrows():
    output_lines.append(f"{row['run_no']:>10}|_{row['median_baseline_shift']:>15.4f}")
output_lines.append("")

output_lines.append("3. Top 5 Runs with Largest Absolute Baseline Shift:")
output_lines.append(f"run_no|Median Shift(ml)")
output_lines.append("-" * 50)
for _, row in top_shifts.iterrows():
    output_lines.append(f"{row['run_no']:>10}|_{row['median_baseline_shift']:>15.4f}")
output_lines.append("")

output_lines.append("4. Temporal Gap Analysis Details:")
output_lines.append(f"run_no|Fraction_number|Action")
output_lines.append("-" * 50)
gap_details = df_baseline[df_baseline['temporal_gap_flag']]
for _, row in gap_details.iterrows():
    output_lines.append(f"{row['run_no']:>10}|_{row['Fraction_number']:>15}|_{row['gap_action']}")
output_lines.append("")

gap_count_total = df_baseline['temporal_gap_flag'].sum()
short_run_count_total = df_baseline['short_run_flag'].sum()

output_lines.append("5. Run Status Counts:")
output_lines.append(f"Runs with Temporal Gaps: {gap_count_total}")
output_lines.append(f"Short Runs (<40 fractions): {short_run_count_total}")
output_lines.append(f"Short Runs Skipped Gaussian Fitting: {short_run_count_total}")

output_file = '/workspace/Step1_BaselineCorrection.txt'
with open(output_file, 'w') as f:
    f.write('\n'.join(output_lines))

print(f"Report saved to {output_file}")

if __name__ == "__main__":
    main()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def gaussian(x, a, x0, sigma):
    return a * np.exp(-((x - x0) ** 2) / (2 * sigma ** 2))

def main():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Step 1: Filter to volume_ml_corrected > 0
    if 'volume_ml_corrected' in df.columns:
        df = df[df['volume_ml_corrected'] > 0]

```

```

elif 'volume_ml' in df.columns:
    df = df[df['volume_ml'] > 0]

# Step 2: Flag UV > 1500/2000. Removed Flow > 4.0 check as 'System_flow_ml_min'
# is missing.
df['is_anomaly'] = (df['UV_1_280_mAU'] > 1500) | (df['UV_2_260_mAU'] > 2000)

flagged_df = df[df['is_anomaly'] == True]

# Step 3: Gaussian Fit
r_squared = 0.0
if len(flagged_df) == 0:
    flagged_df = pd.DataFrame(columns=['Fraction_number', 'UV_1_280_mAU', '
classification'])
    flagged_df['classification'] = 'Detector Artifact'
else:
    mean_frac = flagged_df['Fraction_number'].mean()
    window_start = int(mean_frac - 20)
    window_end = int(mean_frac + 20)

    mask_window = (flagged_df['Fraction_number'] >= window_start) & (
        flagged_df['Fraction_number'] <= window_end)
    window_data = flagged_df[mask_window]

    x_data = window_data['Fraction_number'].values
    y_data = window_data['UV_1_280_mAU'].values

    if len(x_data) < 2:
        flagged_df['classification'] = 'Detector Artifact'
    else:
        p0 = [1000, mean_frac, 10]
        try:
            popt, pcov = curve_fit(gaussian, x_data, y_data, p0=p0)
            y_pred = gaussian(x_data, *popt)
            ss_res = np.sum((y_data - y_pred) ** 2)
            ss_tot = np.sum((y_data - np.mean(y_data)) ** 2)
            r_squared = 1 - (ss_res / ss_tot) if ss_tot != 0 else 0.0

            if r_squared < 0.85:
                flagged_df.loc[window_data.index, 'classification'] = '
Detector Artifact'
            else:
                flagged_df.loc[window_data.index, 'classification'] = 'Valid
Spike'
        except Exception:
            flagged_df.loc[window_data.index, 'classification'] = 'Detector
Artifact'

# Step 4: Threshold Logic
total_rows = len(df)
anomaly_pct = (len(flagged_df) / total_rows) * 100

print(f"Total flagged anomalies: {len(flagged_df)}")

# Step 5: Visualization Logic
# Histogram
plt.figure(figsize=(10, 6))
plt.hist(df['UV_1_280_mAU'], bins=50, alpha=0.7, color='skyblue', edgecolor='
black', label='All Data')

```



```

plt.axvline(x=1500, color='red', linestyle='--', linewidth=2, label='
    Saturation_Threshold_(1500)')
plt.title('UV_Saturation_Distribution')
plt.xlabel('UV_1_280_mAU')
plt.ylabel('Frequency')
plt.legend()
plt.savefig('/workspace/UV_Saturation_Distribution.png')
plt.close()

# Box Plot: Removed 'System_flow_ml_min' as it is missing.
# Check if any flow column exists before plotting
flow_cols = [col for col in df.columns if 'flow' in col.lower()]
if flow_cols:
    plt.figure(figsize=(10, 6))
    plt.boxplot([df[flow_cols[0]], flagged_df[flow_cols[0]]], labels=['Normal',
        'Flagged'])
    plt.title('Flow_Rate_Analysis')
    plt.xlabel('Flow_Category')
    plt.ylabel('Flow_Value')
    plt.savefig('/workspace/Flow_Rate_Capping.png')
    plt.close()
else:
    print("No_flow_columns_found_to_display_in_box_plot.")

# Scatter Plot
scatter_data = flagged_df[['Fraction_number', 'UV_1_280_mAU']]
count_flagged = len(flagged_df)

if len(window_data) > 1 and r_squared > 0:
    plt.figure(figsize=(10, 6))
    plt.scatter(scatter_data['Fraction_number'], scatter_data['UV_1_280_mAU'],
        alpha=0.6, color='orange', label='Flagged_Spikes')
    x_fit = np.linspace(window_start, window_end, 100)
    y_fit = gaussian(x_fit, *popt)
    plt.plot(x_fit, y_fit, 'r--', label='Gaussian_Fit')
    plt.text(mean_frac, max(y_fit)*1.1, f'R_sq={r_squared:.2f}', fontsize=10)

    plt.title(f'Flagged_Spikes_(Count:{count_flagged})')
    plt.xlabel('Fraction_number')
    plt.ylabel('UV_1_280_mAU')
    plt.legend()

    plt.savefig('/workspace/UV_Saturation_Distribution.png')
    plt.close()
else:
    plt.figure(figsize=(10, 6))
    plt.scatter(scatter_data['Fraction_number'], scatter_data['UV_1_280_mAU'],
        alpha=0.6, color='orange', label='Flagged_Spikes')
    plt.title(f'Flagged_Spikes_(Count:{count_flagged})')
    plt.xlabel('Fraction_number')
    plt.ylabel('UV_1_280_mAU')
    plt.legend()

    plt.text(0.05, 0.95, 'Note:Insufficient_data_for_fit', transform=plt.gca
        ().transAxes,
        fontsize=10, verticalalignment='top', bbox=dict(boxstyle='round',
            facecolor='wheat', alpha=0.5))

    plt.savefig('/workspace/UV_Saturation_Distribution.png')

```

```

plt.close()

print(f"Total_rows: {len(df)}")
print(f"Flagged_rows: {len(flagged_df)}")
print(f"Anomaly_percentage: {anomaly_pct:.2f}%")
print(f"Window_data_points: {len(window_data)}")
print(f"R-squared: {r_squared:.4f}")

if __name__ == "__main__":
    main()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Filter dataset where Conc_B_% is not null
df_filtered = df[df['Conc_B_%'].notna()]

# Step 2: Aggregate complete metadata rows by run_no
df_run_agg = df_filtered.groupby('run_no').agg({
    'UV_1_280_mAU': 'mean',
    'Fraction_number': 'first'
}).reset_index()
df_run_agg.columns = ['run_no', 'UV_Mean', 'Fraction_number']

# Step 3: Aggregate complete metadata rows by chromatography_stage
df_stage_agg = df_filtered.groupby('chromatography_stage').agg({
    'run_no': 'count',
    'UV_1_280_mAU': 'mean',
    'Conc_B_%': 'mean',
    'pH_pH': 'mean'
}).reset_index()
df_stage_agg.columns = ['chromatography_stage', 'run_count', 'UV_Mean', 'Conc_B_Mean', 'pH_Mean']

# Step 4: Handle Sparse Features
run_stats = df_filtered.groupby('run_no').agg({
    'Conc_B_%': 'mean',
    'pH_pH': 'mean'
}).reset_index()
run_stats.columns = ['run_no', 'Conc_B_Sparse_Mean', 'pH_Sparse_Mean']

# Merge run stats back to run_agg
df_final = df_run_agg.merge(run_stats, on='run_no', how='left')

# Step 5: Calculate anomaly percentage per run
df_filtered['Any_Sparse_Missing'] = (df_filtered['Conc_B_%'].isna()) | (
    df_filtered['pH_pH'].isna())
missing_counts = df_filtered.groupby('run_no')['Any_Sparse_Missing'].sum().reset_index()
missing_counts.columns = ['run_no', 'missing_count']

missing_counts['anomaly_pct'] = np.round((missing_counts['missing_count'] /
    missing_counts['run_no']) * 100, 2)

```

```

df_final = df_final.merge(missing_counts, on='run_no', how='left')

# Step 6: Impute missing Fraction_number using median for specific run_no
df_final['Fraction_number_imputed'] = df_final.groupby('run_no')['Fraction_number']
    .transform(lambda x: x.fillna(x.median()))
df_final['Fraction_number'] = df_final['Fraction_number_imputed']

# Step 7: Calculate volume_ml_corrected based on anomaly_pct threshold
df_final['volume_ml_corrected'] = np.where(
    df_final['anomaly_pct'] > 5,
    df_final['UV_Mean'] - (df_final['UV_Mean'] * df_final['anomaly_pct'] / 100),
    df_final['UV_Mean']
)

# Step 8: Create Output Text File
output_lines = []
output_lines.append("Step3_RunAggregation.txt")
output_lines.append("=" * 50)
output_lines.append("")

# Summary table
total_runs = len(df_final)
complete_runs = df_final['Conc_B_Sparse_Mean'].notna().sum()
incomplete_runs = total_runs - complete_runs
avg_uv_means = df_final['UV_Mean'].mean()

output_lines.append("Summary Table:")
output_lines.append(f"Total Runs: {total_runs}")
output_lines.append(f"Complete Runs (Both Sparse Features Present): {complete_runs}")
output_lines.append(f"Incomplete Runs: {incomplete_runs}")
output_lines.append(f"Avg UV Means: {avg_uv_means:.2f}")
output_lines.append("")

# List of top 20 run_no with highest anomaly %
top_anomalies = df_final.sort_values('anomaly_pct', ascending=False).head(20)
anomaly_strs = [f"{row['run_no']}: {row['anomaly_pct']}%" for _, row in
    top_anomalies.iterrows()]
output_lines.append("Top 20 run_no with Highest Anomaly%:")
output_lines.append('\n'.join(anomaly_strs))
output_lines.append("")

# Note
output_lines.append("Note:")
output_lines.append("- Fraction_number is explicitly treated as a surrogate for time.")
output_lines.append("- Sparse features (Conc_B_% and pH_pH) are excluded from continuous correlation.")
output_lines.append("- Anomaly_% calculated based on percentage of rows with missing sparse features per run.")
output_lines.append("- Missing Fraction_number values imputed using median per run_no.")
output_lines.append("- volume_ml_corrected adjusted for runs with anomaly_pct > 5%.")

# Write to file
with open('/workspace/Step3_RunAggregation.txt', 'w') as f:
    f.write('\n'.join(output_lines))

```

Listing 3: Code_3